Queen Mary University of London
School of Electronic Engineering and Computer Science

Lab 4 – Memory Management [Answer Sheet]

Part A: Virtual Addresses

Answer the following questions about the behaviour of **stack.c**:

Questions	virtual.c
Q1.1	The C program **virtual.c** outputs virtual addresses in decimal notation written as A*B+C = D. After inspecting the code, what does each of the variables **A**, **B**, **C** and **D** describe (e.g., what would be a good name for each of them)?
Your answer for **Q1.1**	
Q1.2	The C program **virtual.c** outputs virtual addresses both in decimal and hexadecimal notation. One could also manually translate between both (e.g., 0xEFF1 = $14*16^3+ 15*16^2 + 15*16^1 + 1*16^0$). Choose any printed virtual address (copy the related outputs): how would one manually translate the hexadecimal value into a decimal one?
Your answer for **Q1.2**	

Q1.3	**virtual.c** outputs multiple virtual addresses. Going through all printed virtual addresses, what are the lowest and highest virtual addresses? (Copy printed lines of those into your answer box.) Considering that virtual addresses point to a single byte, how many bytes are addressable based on the smallest and highest address? (Does the system have that much RAM?) How many frames do contain some bytes of **arrayOnHeap** and presuming the same frame size, what is the maximal number of frames that could theoretically contain some bytes of **arrayOnHeap**?
Your answer for **Q1.3**	

Part B: Stack

Answer the following questions about the behaviour of **stack.c**:

Questions	stack.c
Q2.1	A stack pointer (SP) is the virtual address of the first byte of the stack. How many bytes does the SP move down per each recursive call? Does this number change when compiling either with the "`-O3`" or "`-O0`" flags?
Your answer for **Q2.1**	
Q2.2	Use a disassembler tool (e.g., Ubuntu/Mac: `objdump` or Mac: `otool`) to analyse the compiled machine code of the program. In which instruction (which should correspond to a disassembler output line) of the machine code does it jump/branch from the **main** function into the **f** function? (Copy that line into your answer box.)
Your answer for **Q2.2**	
Q2.3	Which machine code instructions of the **f** function explain by how much the stack pointer moves exactly as observed in **Q2.1**? (Copy the whole **f** function and prepend a few asterisks (*******) at any explanatory instruction.)

Your answer for **Q2.3**	

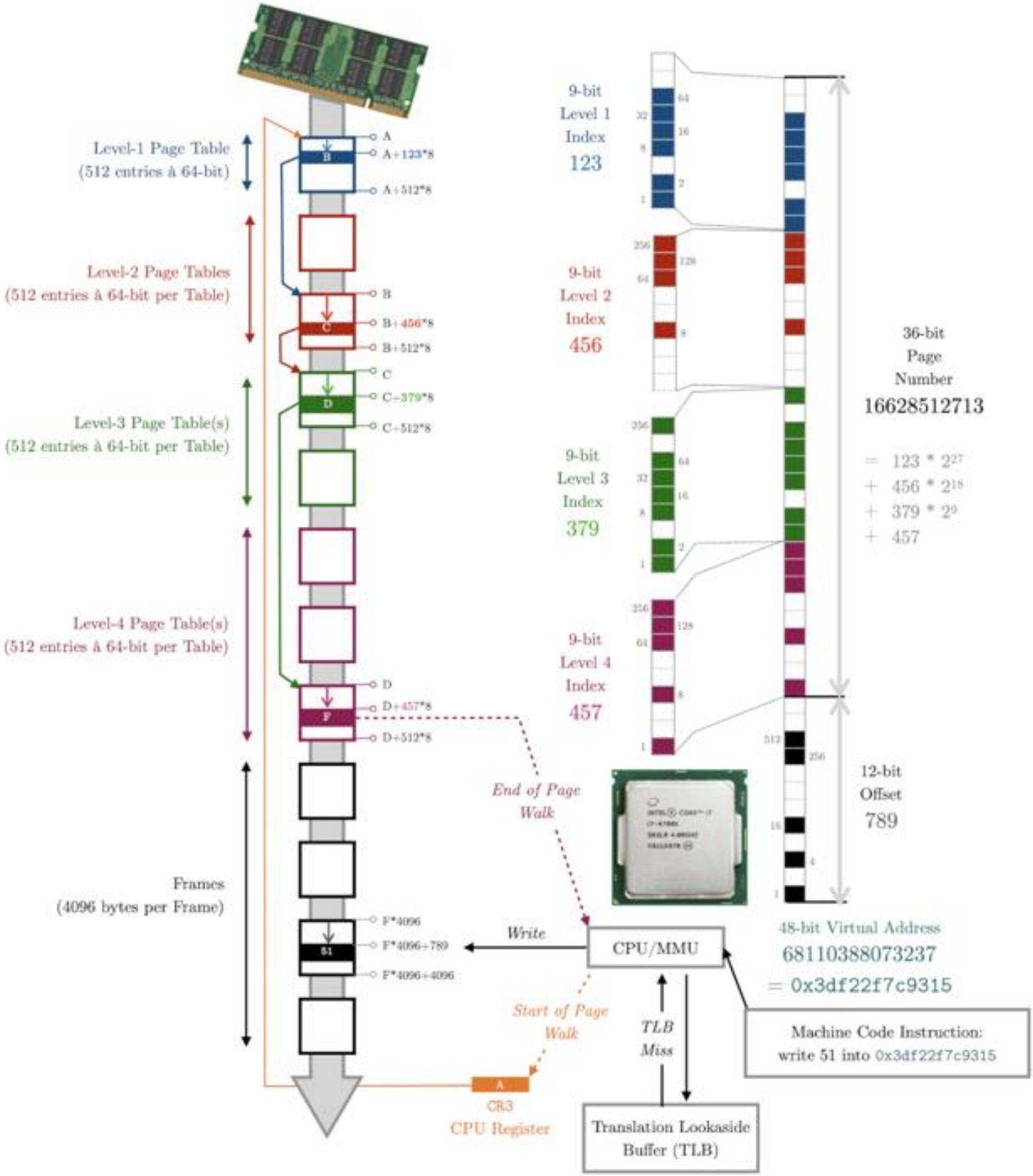
Part C: Page Faults and Address Translation using 4-Level Page Tables

Answer the following questions about the behaviour of **paging.c**:

Questions	paging.c
Q3.1	Use a tool of your choice to monitor (major) page faults caused by a specific process (the code prints some suggested commands). Allocate pages in the simulator and observe if it impacts the number of page faults. How many pages/frames do you need to allocate on the paging simulator running on your system such that a certain number of page faults are attributed to this process? (Ubuntu: 100 Major Page Faults, MacOSX: 100000 “Faults”; alternatively: obvious signs of thrashing.)
Your answer for **Q3.1**	
Q3.2	How much larger would the overhead of a (flat) 1-level page table have been compared to the (hierarchical) 4-level page table scheme/tree? (This is for a similar number of pages/frames allocations as in the answer to **Q3.1**.)
Your answer for **Q3.2**	

Q2.3	What is an example of a set of page numbers (to allocate a frame for) that would lead to the observed number of page tables and subsequent behaviour of **Figure 1** in the **Appendix**? Entering those exemplary page numbers into the simulator, how much memory usage does it report?
Your answer for **Q3.3**	

Appendix – Figure 1 [Example of 4-level Page Tables; explanation in page 8]



Role of OS beyond this example:

- Manages page tables for all processes
- Give CPU/MMU enough information for hardware page walks
  - Updates CR3 register in case of context switches
- Handles page faults
  - TLB miss —> page walk —> page not in memory —> OS
  - TLB hit —> dirty page —> page not in memory —> OS

- *Physical addresses* point directly to a single byte, but are hidden away from user processes both for sake of convenience and security.
- *Virtual addresses* point only indirectly to a single byte, but can be used by user processes as if they were physical addresses on extremely large memory (2^{48} bytes = 256 TB).
- *Address translation* on the CPU/MMU replaces virtual addresses in instructions to physical ones during execution (user process is oblivious to this).
- *Example*:
  - Machine code instructs setting byte value at 48-bit virtual address (`0x3df22f7c9315`) to a new value (51).
    - CPU/MMU splits virtual address into 36-bit page number (16628512713) and 12-bit offset (789) within the page/frame.
  - CPU/MMU checks if TLB knows frame number for page number 16628512713.
    - TLB miss: Translation Lookaside Buffer (TLB) does not know it.
  - Hardware Page Walk by CPU/MMU:
    - *Note:* page number 16628512713 = 123*2^{27} +456*2^{18} + 379*2^{9}+457
    - Split page number into it's four 9-bit indices (123, 456, 379, 457)
    - Read physical address `A` from `CR3` register on the CPU/MMU.
    - Read 8 bytes (64-bit value) starting from physical address `A+123*8`
      - Check least significant 12 bits to rule out OS is needed (not needed here)
        - Interpret the most significant 52 bits as physical address `B`
    - Read 8 bytes (64-bit value) starting from physical address `B+456*8`
      - Check least significant 12 bits to rule out OS is needed (not needed here)
        - Interpret the most significant 52 bits as physical address `C`
    - Read 8 bytes (64-bit value) starting from physical address `C+379*8`
      - Check least significant 12 bits to rule out OS is needed (not needed here)
        - Interpret the most significant 52 bits as physical address `D`
    - Read 8 bytes (64-bit value) starting from physical address `D+457*8`
      - Check least significant 12 bits to rule out OS is needed (not needed here)
        - Interpret the most significant 52 bits as frame number `F`
  - CPU/MMU sets byte at physical address `F*4096+789` to the new value 51.
- *Caveats:*
  - Number of most significant bits for physical addresses is hardware dependent (Typically: AMD uses 52 bits while Intel uses 40 bits)
  - The figure implies A < B < C < D < F and does not depict any gaps for sake of simplicity (it is not impossible, but not likely)
  - There can be more than one TLB cache (akin to other cache hierarchies), but one could think of all the TLB caches as the "TLB". TLB entries can be outdated and even TLB hits may require OS intervention ("dirty" pages etc).
  - On some systems multi-level page tables do not always have to be in physical memory (which requires using virtual instead of physical addresses in entries). Some tables must always be kept in main memory to handle the page faults related to page tables.